CS6220 HW3
Zihe Hao

Question 1: https://github.com/hammer-hao/CS6620-HW3
Question 2:

```
def knn_predict(X_train, y_train, X_test, k, distance_func)
    minHeap = new MinHeap()
    for record in X_train, y_train:
    distance = distance_func(X_test, record)
    if (minHeap.size() < k or -distance > minHeap.top()):
        minHeap.insert(-distance, record)
        if (minHeap.size() > k):
            minHeap.pop()
return average(records in minHeap)
```

Question 3:

Test Point	Distance	k	Class 0 Votes	Class 1 Votes	Prediction
(0.40, 0.45)	Manhattan	1	1	0	0
(0.40, 0.45)	Manhattan	3	1	2	1
(0.40, 0.45)	Manhattan	5	1	4	1
(0.40, 0.45)	Euclidean	1	1	0	0
(0.40, 0.45)	Euclidean	3	1	2	1
(0.40, 0.45)	Euclidean	5	1	4	1
(0.08, 0.78)	Manhattan	1	0	1	1
(0.08, 0.78)	Manhattan	3	2	1	0
(0.08, 0.78)	Manhattan	5	4	1	0
(0.08, 0.78)	Euclidean	1	0	1	1
(0.08, 0.78)	Euclidean	3	2	1	0
(0.08, 0.78)	Euclidean	5	4	1	0
(0.75, -0.15)	Manhattan	1	0	1	1

(0.75, -0.15)	Manhattan	3	1	2	1
(0.75, -0.15)	Manhattan	5	2	3	1
(0.75, -0.15)	Euclidean	1	1	0	0
(0.75, -0.15)	Euclidean	3	1	2	1
(0.75, -0.15)	Euclidean	5	2	3	1

(c) Effect of k and distance_func:

Both k and the distance metric can affect predictions. For the effect of k: for (0.40, 0.45), the prediction changes from class 0 at k=1 to class 1 at k=3 and k=5. For (0.08, 0.78), it changes from class 1 to class 0. For the effect of distance: For (0.75, -0.15) at k=1, Manhattan predicts class 1, whereas Euclidean predicts class 0.

Although k has a larger effect on this dataset, the choice of distance metric can also change the nearest neighbor and predicted class.

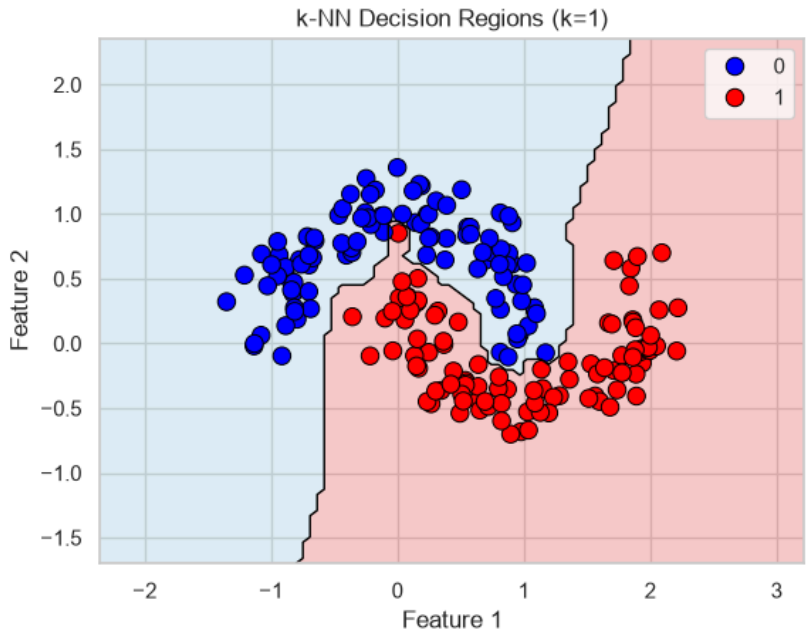
(d)

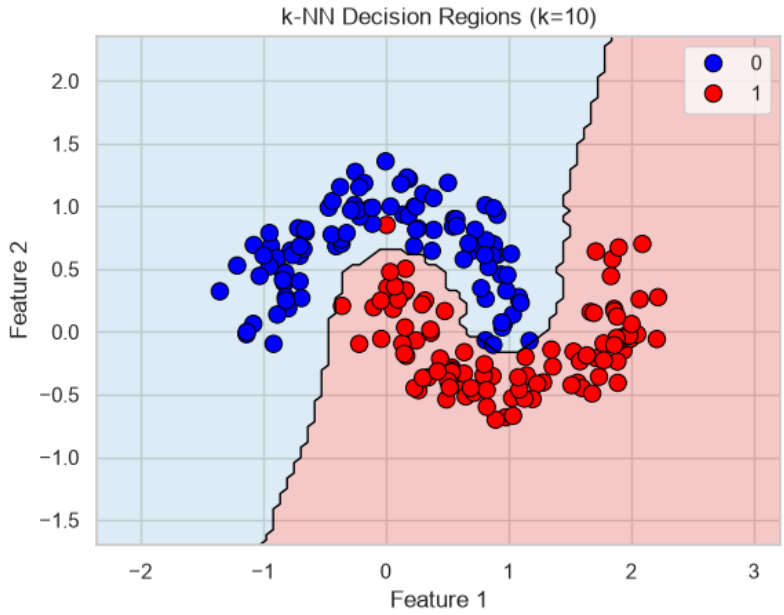
Consider a test point P = (0, 0) and two training points:

- A = (2, 0), class 0
- B = (1.1, 1.1), class 1

Manhattan distance selects A because 2.00 is smaller than 2.20, while Euclidean distance selects B because 1.56 is smaller than 2.00. This demonstrates that the distance metric can change which training point is considered the nearest neighbor, resulting in different k-NN classifications.

Question 4:





K = 1 definitely has a higher variance. Any single point flipping between 0 and 1 can affect our model to generate completely opposite predictions for the region around it. Meanwhile, for K = 10, the variance is small because even if there's one opposite-labeled point that has "infiltrated" the other region, this will not likely change the prediction because the majority of near points still vote against it.

Question 5:

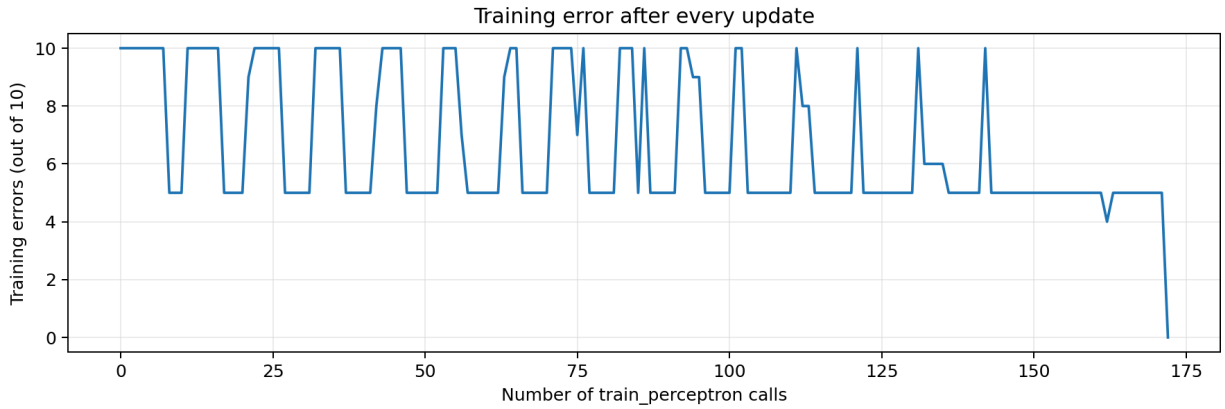
a. As we decrease k, we increase the variance of the model, meaning that the model has more freedom to closely match whatever point we have given it in the training data. For the Knn model, decreasing k means that we look at less and less points around the given point in the training data, i.e. the original point itself has more and more power to influence the prediction, causing us to have closer and closer to perfect predictions.
b. It is possible to reach zero training error. All we have to do is to set k=1 and we will predict all training data based on itself, meaning that we are directly comparing the training Y against itself, which will always result in error zero.
c. If our training data has X = [0, 0], [0, 0], and Y = [0, 1]. It is impossible to get training error zero, because whichever value the model predicts for the input [0, 0], there will always be some error.
d. If we increase k from size 1 to size n, we will essentially just be predicting the mean value of the training dataset, as any input results in the same n nearest neighbors. Both training and test error will be very high.

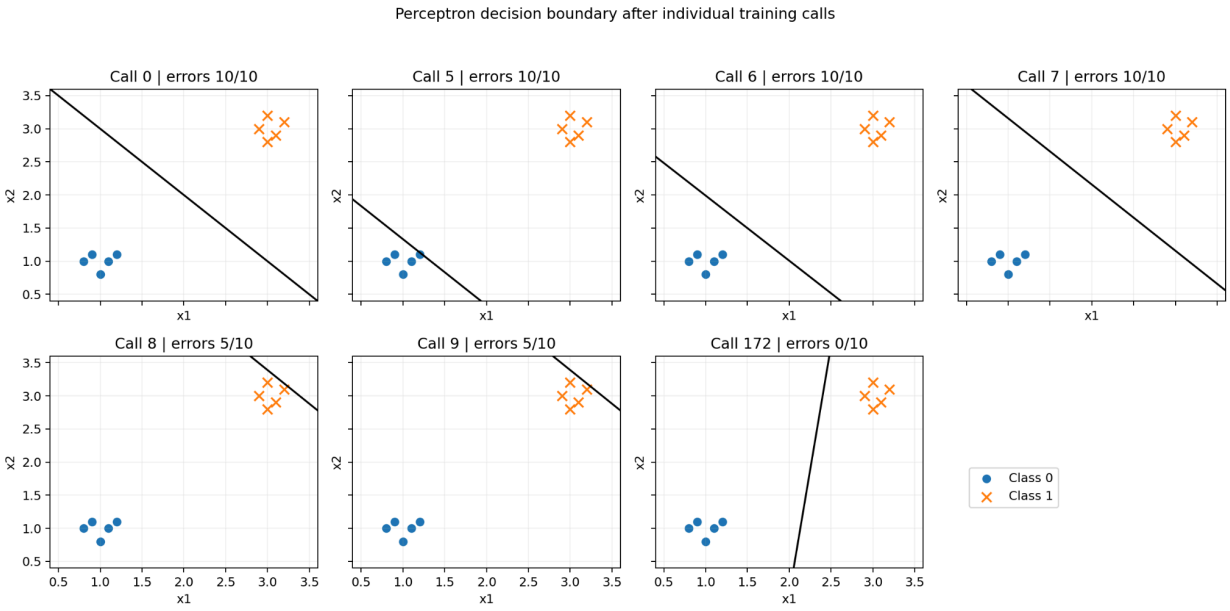
Question 6:

```
FUNCTION train_perceptron(x1, x2, y, learning_rate, w1, w2, b):
    z = w1*x1 + w2*x2 + b
    prediction = 1 if z >= 0 else 0
    error = y - prediction
    w1 = w1 + learning_rate * error * x1
    w2 = w2 + learning_rate * error * x2
    b = b + learning_rate * error
    RETURN w1, w2, b
```

```
FUNCTION train_neuron(x, y, learning_rate, w, b):
    z = w*x + b
    prediction = activation(z)
    cost_gradient = cost_derivative(prediction, y)
    activation_gradient = activation_derivative(z)
    delta = cost_gradient * activation_gradient
    w = w - learning_rate * delta * x
    b = b - learning_rate * delta
    RETURN w, b
```

Question 7:





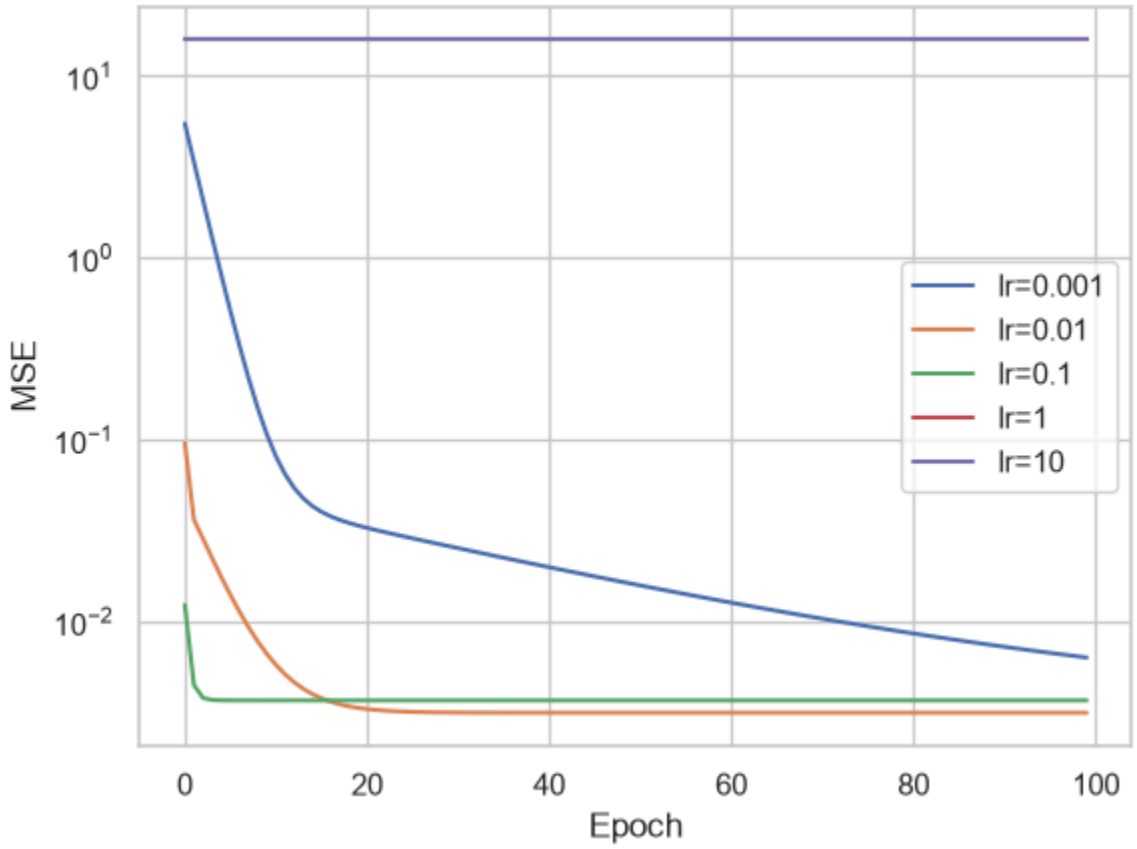
Starting with weights (-1, -1) and bias 4, the perceptron initially misclassified all 10 training records. Using a learning rate of 0.1, I updated the parameters one record at a time and evaluated the full dataset after each update. Although the training error fluctuated, the perceptron reached zero errors after 172 updates. The final decision boundary perfectly separates the two classes, demonstrating that the perceptron can learn a linearly separable dataset.

Question 8:

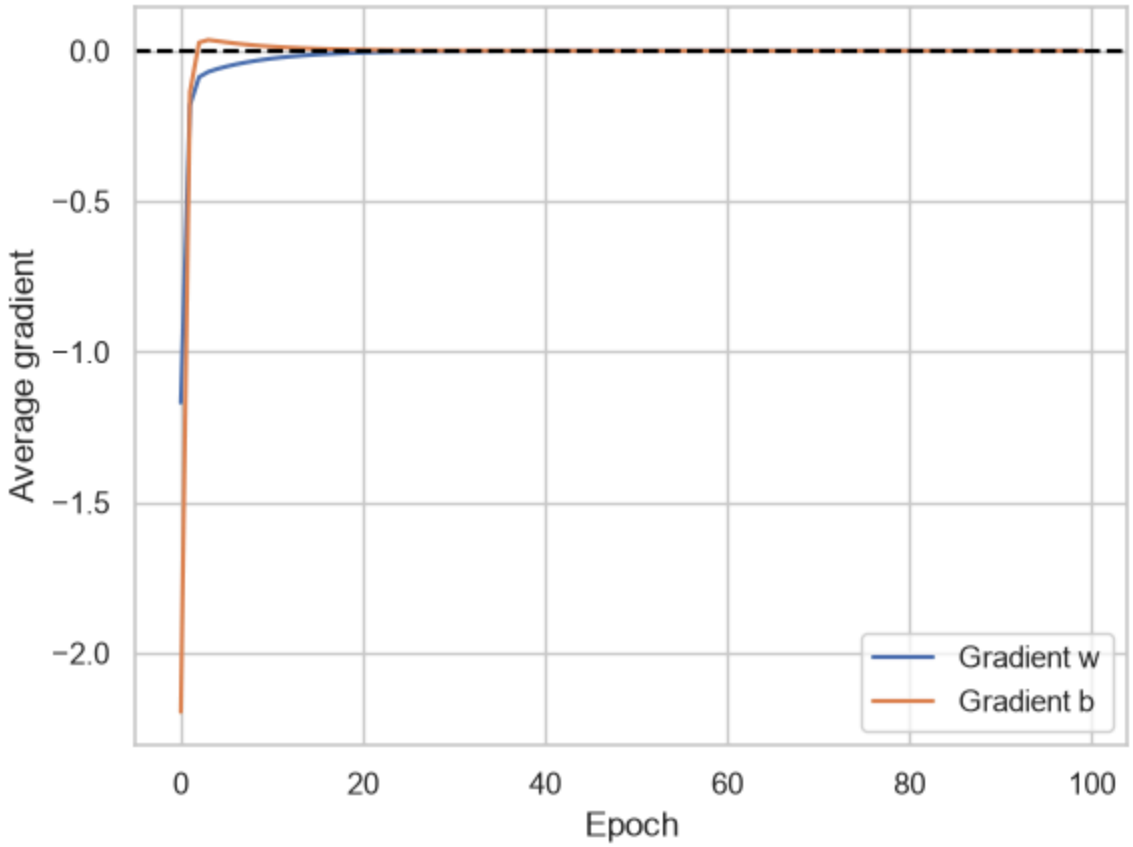
After adding (1.0, 1.0, 1), the dataset is no longer linearly separable because the new class 1 point lies within the class 0 cluster. The perceptron achieves a minimum training error of 1/11 but cannot perfectly classify all records. Therefore, the algorithm does not converge to a zero-error solution, and its weights and decision boundary continue to fluctuate during training.

Question 9:

a. For the animation, see the notebook on the GitHub repository.
b. The RELU activation function seemed to work the best, and the squared error cost function worked better than the cross-entropy function. The RELU activation function resulted in a much lower error very early on in the training, while sigmoid and tanh had trouble converging, and their training seemed to have came to a halt with vanishing gradient while the error is still quite high. The squared error cost function worked better since it better captured the nature of the model we are trying to predict than cross-entropy, which was better for models used for classification.
c. Observing the plot below, lower learning rates often are associated with slower convergence, but overall higher model quality when the convergence does arrive. Furthermore, setting the learning rate to something extreme like 10 will cause the model to take huge steps and therefore never converge.



d. For lr = 0.01 and activation = “relu”:



The gradient was initially very far from zero. For w it was around -1.2 and for b it was around -2.2. However, after a few epochs, those gradients quickly approached zero. This is because we are using linear activation function with the squared error cost function. More specifically, the mean squared error cost function's gradient is linear in the prediction error. Therefore, as the model learns and the cost gets lower, the magnitude of the gradient decreases as well.

Question 10:

The multi-layer perceptron has 5 layers in total: 4 hidden layers and 1 output layer, excluding the input layer. Each hidden layer contains 32 neurons, while the output layer contains 10 neurons corresponding to the 10 MNIST digit classes. In the provided configuration, all five layers use the sigmoid activation function. The network uses mean squared error (MSE) as its cost function and stochastic gradient descent (SGD) for optimization.

Question 11:

The plot shows how the magnitude of the gradient changes across training epochs for each layer of the neural network.

Each curve represents one layer, with the x-axis showing epochs and the y-axis showing the normalized L1 norm of the gradient on a logarithmic scale. The shape of each curve tells us how the gradient changes during training, while the differences between curves indicate how effectively gradients propagate through the network. In particular, the earlier layers are expected to have much smaller gradients than the later layers, indicating a vanishing gradient problem caused by repeated sigmoid derivatives during backpropagation. This makes learning in the earlier layers extremely slow.

Question 12:

Changing the activation and cost functions affects how gradients propagate through the network. With sigmoid activations and MSE, gradients tend to become very small, particularly in the earlier layers, because sigmoid derivatives are at most 0.5^2 = 0.25 and the MSE gradient can introduce additional shrinking.

Switching to cross-entropy with an identity output activation generally produces stronger gradients at the output layer because cross-entropy avoids the extra sigmoid derivative at the output. However, sigmoid hidden layers can still experience vanishing gradients, i.e. the root problem still persists. Replacing hidden sigmoid activations with ReLU generally improves gradient propagation because ReLU has a derivative of 1 for positive inputs. Therefore, I would expect the network to learn best using ReLU in the hidden layers, identity at the output, and cross-entropy cost, as this combination generally supports faster convergence and better classification performance.